



GESTION DE MIGRATION

INTRODUCTION

Les migrations sont comme un contrôle de version pour votre base de données, permettant à votre équipe de définir et de partager la définition du schéma de la base de données de l'application.

La façade Laravel **Schema** fournit une prise en charge agnostique des bases de données pour la création et la manipulation de tableaux dans tous les systèmes de bases de données pris en charge par Laravel. En général, les migrations utilisent cette façade pour créer et modifier les tables et les colonnes de la base de données.

GÉNÉRER DES MIGRATIONS

Vous pouvez utiliser la commande Artisan **make:migration** pour générer une migration de base de données. La nouvelle migration sera placée dans votre répertoire **database/migrations**. Chaque nom de fichier de migration contient un horodatage qui permet à Laravel de déterminer l'ordre des migrations :

```
php artisan make:migration create_flights_table
```

RÉDUIRE LES MIGRATIONS

À mesure que vous construisez votre application, vous pouvez accumuler de plus en plus de migrations au fil du temps. Cela peut conduire à ce que votre répertoire de base de **données/migrations** soit gonflé par des centaines de migrations potentielles. Si vous le souhaitez, vous pouvez "Réduire" vos migrations dans un seul fichier SQL en utilisant la commande `schema:dump` :

```
php artisan schema:dump
```

RÉDUIRE LES MIGRATIONS

Lorsque vous exécutez cette commande, Laravel écrit un fichier "**schema**" dans le répertoire **database/schema** de votre application. Le nom du fichier schema correspondra à la connexion à la base de données.

Lorsque vous tentez de migrer votre base de données et qu'aucune autre migration n'a été exécutée, Laravel exécutera d'abord les instructions SQL du fichier de schéma. Après avoir exécuté les instructions du fichier de schéma, Laravel exécutera toutes les migrations restantes qui ne faisaient pas partie du fichier schema.

STRUCTURE DE LA MIGRATION

Une classe de migration contient deux méthodes : **up** et **down**. La méthode **up** est utilisée pour ajouter de nouvelles tables, colonnes ou index à votre base de données, tandis que la méthode **down** doit inverser les opérations effectuées par la méthode **up**.

STRUCTURE DE LA MIGRATION

```
public function up()  
{  
    Schema::create('flights', function (Blueprint $table) {  
        $table->id();  
        $table->string('name');  
        $table->string('airline');  
        $table->timestamps();  
    });  
}
```

STRUCTURE DE LA MIGRATION

```
public function down()  
{  
    Schema::drop('flights');  
}
```


CONFIGURATION DE LA CONNEXION DE LA MIGRATION

Si votre migration doit interagir avec une connexion à une base de données autre que la connexion par défaut de votre application, vous devez définir la propriété **\$connection** de votre migration :

```
protected $connection = 'pgsql';
```

```
public function up()  
{  
    //  
}
```

EXÉCUTION DES MIGRATIONS

Pour exécuter toutes vos migrations en cours, exécutez la commande Artisan **migrate** :

```
php artisan migrate
```

Si vous souhaitez voir quelles migrations ont été exécutées jusqu'à présent, vous pouvez utiliser la commande Artisan **migrate:status** :

```
php artisan migrate:status
```

ANNULER LA DERNIÈRE OPÉRATION DE MIGRATION

Pour annuler la dernière opération de migration, vous pouvez utiliser la commande Artisan **rollback**. Cette commande annule le dernier "lot" de migrations, qui peut inclure plusieurs fichiers de migration :

```
php artisan migrate:rollback
```

ANNULER LA DERNIÈRE OPÉRATION DE MIGRATION

La commande **migrate:reset** annulera toutes les migrations de votre application :

```
php artisan migrate:reset
```

ANNULATION ET MIGRATION

La commande **migrate:refresh** annulera toutes vos migrations et exécutera ensuite la commande migrate. Cette commande recrée effectivement l'ensemble de votre base de données :

```
php artisan migrate:refresh
```

SUPPRIMER TOUTES LES TABLES ET MIGRER

La commande **migrate:fresh** va supprimer toutes les tables de la base de données et ensuite exécuter la commande migrate :

```
php artisan migrate:fresh
```



TABLEAUX

CRÉATION DE TABLEAUX

Pour créer une nouvelle table de base de données, utilisez la méthode **create** de la façade **Schema**. La méthode **create** accepte deux arguments : le premier est le nom de la table, tandis que le second est une fermeture qui reçoit un objet **Blueprint** qui peut être utilisé pour définir la nouvelle table.

CRÉATION DE TABLEAUX

```
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email');
    $table->timestamps();
});
```

VÉRIFICATION DE L'EXISTENCE D'UNE TABLE OU D'UNE COLONNE

Vous pouvez vérifier l'existence d'une table ou d'une colonne à l'aide des méthodes **hasTable** et **hasColumn** :

```
if (Schema::hasTable('users')) {  
    // The "users" table exists...  
}  
  
if (Schema::hasColumn('users', 'email')) {  
    // The "users" table exists and has an "email" column...  
}
```

MISE À JOUR DES TABLEAUX

La méthode **table** de la façade **Schema** peut être utilisée pour mettre à jour des tables existantes. Comme la méthode `create`, la méthode `table` accepte deux arguments : le nom de la table et une fermeture qui reçoit une instance `Blueprint` que vous pouvez utiliser pour ajouter des colonnes ou des index à la table :

```
Schema::table('users', function (Blueprint $table) {  
    $table->integer('votes');  
});
```

RENOMMER / SUPPRIMER DES TABLES

Pour renommer une table de base de données existante, utilisez la méthode **rename** :

```
Schema::rename($from, $to);
```

Pour supprimer une table existante, vous pouvez utiliser les méthodes **drop** ou **dropIfExists** :

```
Schema::drop('users');
```

```
Schema::dropIfExists('users');
```



COLONNES

CRÉATION DE COLONNES

La méthode **table** de la façade Schema peut être utilisée pour mettre à jour des tables existantes. La méthode table accepte deux arguments : le nom de la table et une fermeture **Blueprint** que vous pouvez utiliser pour ajouter des colonnes à la table :

```
Schema::table('users', function (Blueprint $table) {  
    $table->integer('votes');  
});
```

Voilà la liste de types de colonnes : [Laravel Docs](#)

MODIFICATEURS DE COLONNE

Il existe plusieurs "modificateurs" de colonne que vous pouvez utiliser lorsque vous ajoutez une colonne à une table de base de données. Par exemple, pour rendre la colonne "nullable", vous pouvez utiliser la méthode **nullable** :

```
Schema::table('users', function (Blueprint $table) {  
    $table->string('email')->nullable();  
});
```

MODIFICATEURS DE COLONNE

- >**after('colonne')** Place la colonne "après" une autre colonne (MySQL).
- >**autoIncrement()** Définit les colonnes INTEGER comme auto-incrémentées (clé primaire).
- >**charset('utf8mb4')** Spécifier un jeu de caractères pour la colonne (MySQL).
- >**collation('utf8mb4_unicode_ci')** Spécifiez une collation pour la colonne (MySQL/PostgreSQL/SQL Server).
- >**comment('mon commentaire')** Ajoute un commentaire à une colonne (MySQL/PostgreSQL).
- >**default(\$value)** Spécifie une valeur "par défaut" pour la colonne.
- >**first()** Place la colonne "en premier" dans la table (MySQL).
- >**from(\$integer)** Définit la valeur de départ d'un champ auto-incrémenté (MySQL / PostgreSQL).
- >**invisible()** Rend la colonne "invisible" aux requêtes SELECT * (MySQL).

MODIFICATEURS DE COLONNE

- >**nullable(\$value = true)** Permet d'insérer des valeurs NULL dans la colonne.
- >**storedAs(\$expression)** Créer une colonne générée stockée (MySQL / PostgreSQL).
- >**unsigned()** Définit les colonnes INTEGER comme UNSIGNED (MySQL).
- >**useCurrent()** Définit les colonnes TIMESTAMP pour utiliser CURRENT_TIMESTAMP comme valeur par défaut.
- >**useCurrentOnUpdate()** Configure les colonnes TIMESTAMP pour qu'elles utilisent CURRENT_TIMESTAMP lorsqu'un enregistrement est mis à jour.
- >**virtualAs(\$expression)** Créer une colonne virtuelle générée (MySQL).
- >**generatedAs(\$expression)** Crée une colonne d'identité avec les options de séquence spécifiées (PostgreSQL).
- >**always()** Définit la priorité des valeurs de séquence sur les entrées pour une colonne d'identité (PostgreSQL).
- >**isGeometry()** Définit le type de colonne spatiale à géométrie - le type par défaut est géographie (PostgreSQL).

EXPRESSIONS PAR DÉFAUT

Le modificateur **default** accepte une valeur ou une instance Illuminate\Database\Query\Expression.

L'utilisation d'une instance d'expression empêchera

Laravel de mettre la valeur entre guillemets et vous permettra d'utiliser des fonctions spécifiques à la base de données.

```
Schema::table('users', function (Blueprint $table) {  
    $table->string('city')->default('Rabat');  
});
```

MISE À JOUR DES ATTRIBUTS DES COLONNES

La méthode **change** vous permet de modifier le type et les attributs des colonnes existantes.

```
Schema::table('users', function (Blueprint $table) {  
    $table->string('name', 50)->nullable()->change();  
});
```

SUPPRIMER DES COLONNES

Pour supprimer une colonne, vous pouvez utiliser la méthode **dropColumn** :

```
Schema::table('users', function (Blueprint $table) {  
    $table->dropColumn('votes');  
});
```



INDEX

CRÉATION D'INDEX

Le constructeur de schémas de Laravel prend en charge plusieurs types d'index. L'exemple suivant crée une nouvelle colonne email et spécifie que ses valeurs doivent être uniques. Pour créer l'index, nous pouvons enchaîner la méthode **unique** sur la définition de la colonne :

```
Schema::table('users', function (Blueprint $table) {  
    $table->string('email')->unique();  
});
```

CRÉATION D'INDEX

Vous pouvez également créer l'index après avoir défini la colonne. Pour ce faire, vous devez appeler la méthode **unique** sur le blueprint du constructeur de schéma. Cette méthode accepte le nom de la colonne qui doit recevoir un index unique :

```
Schema::table('users', function (Blueprint $table) {  
    $table->unique('email');  
});
```

TYPES D'INDICES DISPONIBLES

- >**primary('id')** ; Ajoute une clé primaire.
- >**primary(['id', 'parent_id'])** ; Ajoute des clés composées.
- >**unique('email')** ; Ajoute un index unique.
- >**index('state')** ; Ajoute un index.
- >**fullText('body')** ; Ajoute un index de texte complet (MySQL/PostgreSQL).
- >**fullText('body')->language('english')** ; Ajoute un index en texte intégral de la langue spécifiée (PostgreSQL).
- >**spatialIndex('location')** ; Ajoute un index spatial (sauf SQLite).

ABANDON DES INDEX

>dropPrimary('users_id_primary') ; Supprime une clé primaire de la table "users".

>dropUnique('users_email_unique') ; Supprime un index unique de la table "users".

>dropIndex('geo_state_index') ; Supprime un index de base de la table "geo".

>dropFullText('posts_body_fulltext') ; Supprimez un index de texte complet de la table "posts".

>dropSpatialIndex('geo_location_spatialindex') ;
Supprime un index spatial de la table "geo" (sauf SQLite).

CONTRAINTES DE CLÉ ÉTRANGÈRE

Laravel permet également de créer des contraintes de clés étrangères. Exemple, définissons une colonne *user_id* sur la table *posts* qui fait référence à la colonne *id* sur la table *users* :

```
Schema::table('posts', function (Blueprint $table) {  
    $table->unsignedBigInteger('user_id');  
  
    $table->foreign('user_id')->references('id')->on('users');  
});
```

CONTRAINTES DE CLÉ ÉTRANGÈRE

La méthode **foreignId** crée une colonne équivalente, tandis que la méthode **constrained** utilisera des conventions pour déterminer le nom de la table et de la colonne référencée.

```
Schema::table('posts', function (Blueprint $table) {  
    $table->foreignId('user_id')->constrained();  
  
    // spécifier le nom de la table  
    $table->foreignId('user_id')->constrained('users');  
});
```

CONTRAINTES DE CLÉ ÉTRANGÈRE

Vous pouvez également spécifier l'action souhaitée pour les propriétés "on delete" et "on update" de la contrainte :

```
$table->foreignId('user_id')  
    ->constrained()  
    ->onUpdate('cascade')  
    ->onDelete('cascade');
```

CONTRAINTES DE CLÉ ÉTRANGÈRE

- >**cascadeOnUpdate()** ; Les mises à jour doivent être effectuées en cascade.
- >**restrictOnUpdate()** ; Les mises à jour doivent être restreintes.
- >**cascadeOnDelete()** ; Les suppressions doivent être effectuées en cascade.
- >**restrictOnDelete()** ; Les suppressions doivent être restreintes.
- >**nullOnDelete()** ; Les suppressions doivent mettre la valeur de la clé étrangère à null.